Now, we can specify the exact behavior we want, with no ambiguity:

12-creating-custom-matchers/05/custom_matchers.rb

```
# Strict: requires both conditions to be satisfied
expect('blazingly').to(
  start_with_something_besides('a').and \
    end_with_something_besides('z')
)

# Permissive: requires at least one condition to be satisfied
expect('absolutely').to(
  start_with_something_besides('a').or \
    end_with_something_besides('z')
)
```

The techniques we've seen so far—helper methods, matcher aliases, and negated matchers—are all about exposing existing matchers with new names. In the next section, we'll take the next logical step: creating a new matcher that's not based on an existing one.

Using the Matcher DSL

In *Building an App With RSpec 3*, you built an expense-tracking API. If this grows to include expense accounts, you'd end up writing specs that check account balances. In this section, we're going to build a custom have_a_balance_of matcher that helps with those expectations. Here's what the matcher will eventually look like:

12-creating-custom-matchers/06/custom_matcher/spec/initial_account_spec.rb

```
expect(account).to have_a_balance_of(30)
```

Unlike most of the other examples in the book, the code snippets in this section aren't intended for you to type in as you read. We want to focus on just the matcher, without cluttering the examples with all the supporting code. If you *do* want to run these snippets on your machine, you can get the class we're testing (as well as the matcher and some specs) from the book's source code.[2]

There are two ways to build a matcher like the one we just showed you:

Using the matcher DSL

For most needs, RSpec provides a domain-specific language (DSL) for defining custom matchers.

Creating a Ruby class

Any Ruby class can define a matcher if it implements the *matcher protocol*.[3]

2. https://github.com/rspec-3-book/book-code/blob/v1.0/12-creating-custom-matchers/06/custom_matcher/lib/account.rb
3. http://rspec.info/documentation/3.6/rspec-expectations/RSpec/Matchers/MatcherProtocol.html